

AI SCIENTIST

FASE 2 | AULA 02 -

PROCESSAMENTO DISTRIBUÍDO: HADOOP E SPARK

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 7
MERCADO, CASES E TENDÊNCIAS 14
O QUE VOCÊ VIU NESTA AULA? 16
REFERÊNCIAS 17

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O QUE VEM POR AÍ?

À medida que o volume, a velocidade e a variedade dos dados crescem, torna-se inviável processar grandes massas de informação em um único servidor. É nesse contexto que o **processamento distribuído** se consolida como um pilar essencial da Arquitetura de Big Data. Nesta aula, você será convidado(a) a refletir sobre como organizações conseguem analisar petabytes de dados, em tempo hábil, utilizando clusters de máquinas comuns, de forma escalável e tolerante a falhas.

Na sequência, vamos explorar os fundamentos e o funcionamento de dois ecossistemas centrais desse cenário: o **Apache Hadoop**, que revolucionou o armazenamento e o processamento distribuído em larga escala, e o **Apache Spark**, que ampliou esse modelo ao trazer processamento em memória, maior performance e suporte nativo a workloads analíticos, streaming e Machine Learning. Ao final, você entenderá como essas tecnologias se complementam, quando utilizá-las e por que elas são tão estratégicas para arquiteturas modernas de Big Data.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

HANDS ON

Neste hands on, você irá **conectar na prática os conceitos de Arquitetura de Dados e Processamento Distribuído**, dando continuidade direta ao exercício de **Data Lake vs Data Warehouse**. A proposta é mostrar como os dados armazenados em um Data Lake podem ser processados em larga escala utilizando um motor distribuído, sem a necessidade de movimentação para sistemas intermediários. Para isso, será utilizado o **Databricks**, que fornece um ambiente gerenciado para execução do **Apache Spark**. Durante a videoaula, você acompanhará desde a criação da sessão até a execução de transformações e consultas analíticas, visualizando claramente o papel do processamento distribuído dentro de uma arquitetura Lakehouse.

O exercício foi estruturado de forma incremental, permitindo que você entenda não apenas o código, mas também **o motivo de cada etapa existir dentro da arquitetura**. Começamos com a criação do cluster (sessão Spark), avançamos para a leitura de dados diretamente do Data Lake, aplicamos transformações distribuídas e finalizamos com a gravação e o consumo analítico dos dados, simulando uma camada de Data Warehouse construída sobre o próprio Data Lake.

Passo a Passo — Preparando o Ambiente

Passo 1 — Acessar o Databricks Free

1. Acesse: https://community.cloud.databricks.com.
2. Faça login ou crie uma conta gratuita.
3. Após o login, você será direcionado ao **Workspace.**

Passo 2 — Criar um Notebook

1. No menu lateral esquerdo, clique em **Workspace .**
2. Clique com o botão direito → **Create** → **Notebook.**

Nomeie o notebook, por exemplo: `HandsOn_Spark_Databricks`

Selecione:

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

4. Language: Python.
5. Cluster: Starter (padrão do Community Edition).

Passo 3 — Validar o Ambiente

- Execute a célula abaixo para validar o ambiente Spark: Python.
- Spark.

Se o objeto `SparkSession` for exibido, o ambiente está pronto.

Passo 4 — Leitura de dados do Data Lake

Neste passo, os dados são lidos diretamente do Data Lake em formato CSV. O Spark realiza automaticamente a divisão dos dados entre os nós do cluster, evidenciando o conceito de paralelismo transparente.

```
df_raw = spark.read.format("csv") \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .load("/databricks-datasets/learning-spark-
v2/people/people.csv")

df_raw.display()
```

Código - fonte 1: CSV em Python
Fonte: Elaborado pelo autor (2026)

Passo 5 — Transformações distribuídas

Com os dados carregados, aplicamos uma transformação típica de negócio, simulando a passagem da camada raw para uma camada refined ou curated dentro do Data Lake.

```
from pyspark.sql.functions import avg

df_transformado = df_raw.groupBy("occupation") \
    .agg(avg("salary").alias("media_salarial"))

df_transformado.display()
```

Código - fonte 2: Python
Fonte: Elaborado pelo autor (2026)

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Passo 6 — Persistência analítica em Delta Lake

O resultado é gravado em formato Delta Lake, representando uma camada analítica que, em arquiteturas tradicionais, estaria associada ao Data Warehouse, agora integrada ao Data Lake.

```
df_transformado.write.format("delta") \
    .mode("overwrite") \
    .save("/tmp/delta/media_salarial_por_ocupacao")
```

Código - fonte 3: Biblioteca PySpark - Python
Fonte: Elaborado pelo autor (2026)

Passo 7 — Consumo analítico com Spark SQL

Por fim, os dados tratados são consultados via Spark SQL, demonstrando como análises estruturadas podem ser realizadas diretamente sobre o Data Lake.

```
SELECT occupation, media_salarial
FROM delta.`/tmp/delta/media_salarial_por_ocupacao`
ORDER BY media_salarial DESC;
```

Código - fonte 4: Spark SQL
Fonte: Elaborado pelo autor (2026)

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

SAIBA MAIS

Processamento distribuído: Hadoop e Spark

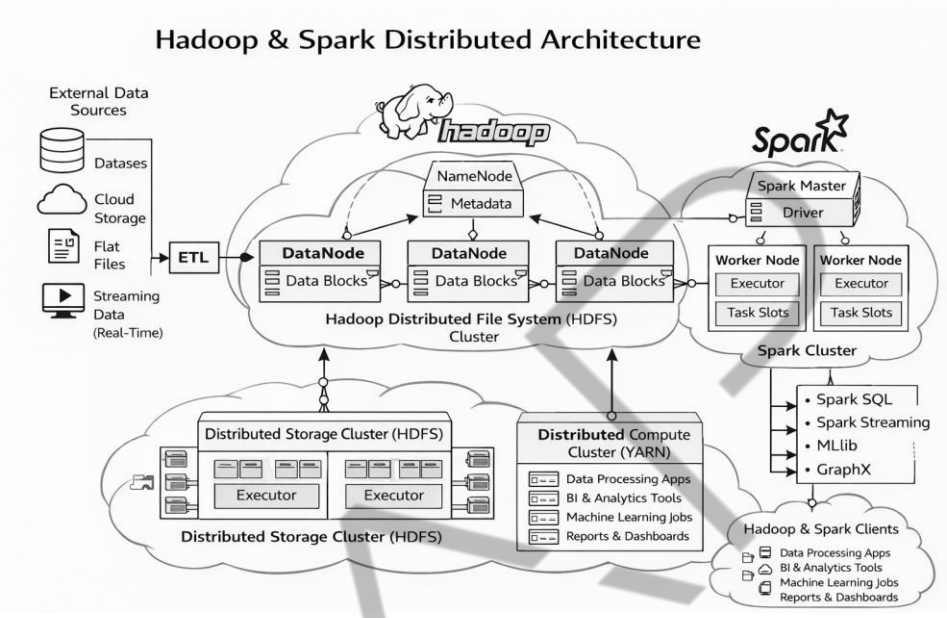


Figura 1 - Hadoop e Spark
Fonte: Elaborado pelo autor (2026)

O crescimento exponencial dos dados nas organizações tornou inviável o modelo tradicional de processamento centralizado. À medida que volumes maiores passaram a ser gerados por sistemas transacionais, sensores, redes sociais e aplicações digitais, ficou claro que um único servidor não seria capaz de armazenar, processar e analisar essas informações dentro de janelas de tempo aceitáveis. O **processamento distribuído** surge exatamente para resolver esse problema, permitindo que tarefas sejam divididas e executadas em paralelo por diversos nós de um cluster, garantindo escalabilidade, resiliência e melhor aproveitamento de recursos computacionais.

Nesse modelo, os dados são fragmentados em partes menores e distribuídos entre várias máquinas, que trabalham de forma coordenada. Caso um nó falhe, o sistema continua operando, redistribuindo automaticamente as tarefas. Essa abordagem está diretamente ligada à ideia de **escala horizontal**, amplamente utilizada em arquiteturas de Big Data, em que se adicionam novos servidores ao

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

cluster conforme a demanda cresce, em vez de investir em máquinas cada vez mais potentes e custosas.

Apache Hadoop

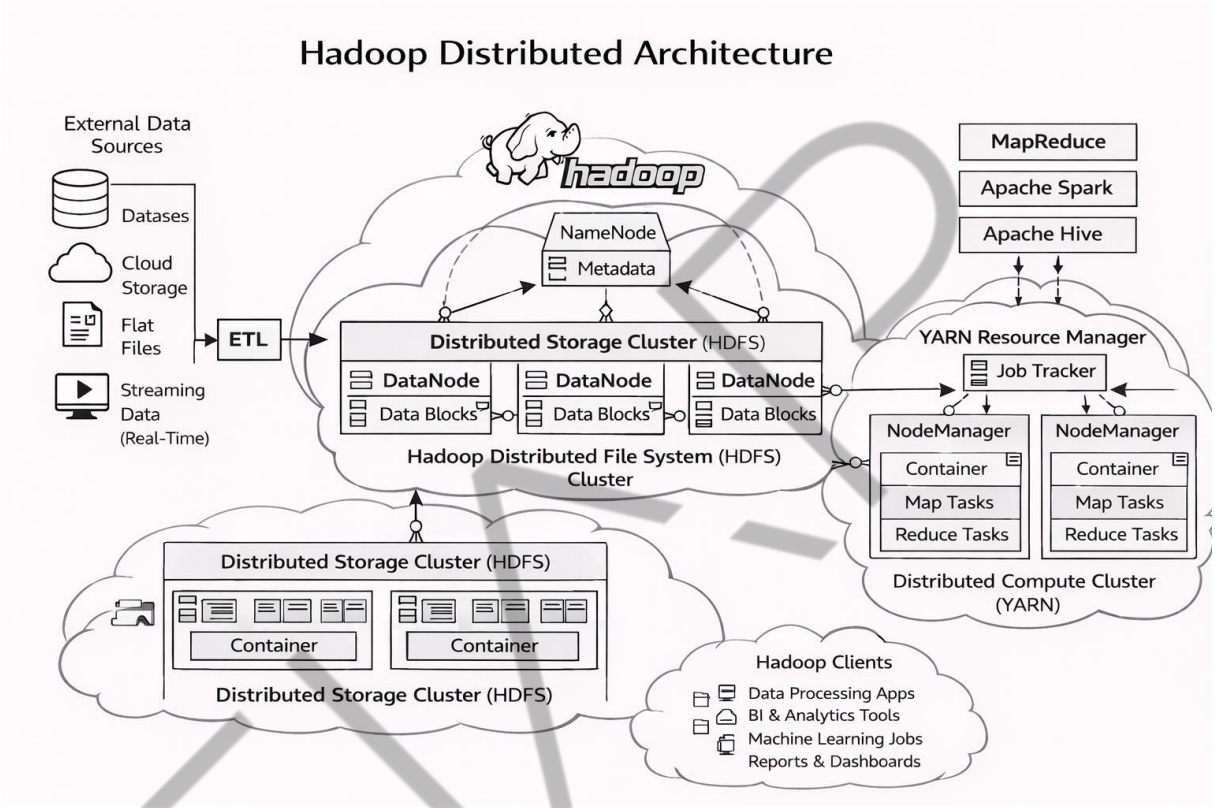


Figura 2 - Apache Hadoop
Fonte: Elaborado pelo autor (2026)

O **Apache Hadoop** foi uma das primeiras plataformas a viabilizar o processamento distribuído em larga escala de forma acessível. Ele introduziu um ecossistema capaz de armazenar grandes volumes de dados e processá-los de maneira paralela, utilizando hardware commodity. O Hadoop é composto, principalmente, por três pilares fundamentais: HDFS, YARN e MapReduce.

O **HDFS (Hadoop Distributed File System)** é o sistema de arquivos distribuído responsável pelo armazenamento. Ele divide arquivos grandes em blocos menores, distribuídos e replicados entre os nós do cluster. Essa replicação garante tolerância a falhas, pois, caso um nó fique indisponível, outra réplica do bloco pode ser utilizada sem interrupção do processamento.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O **MapReduce** é o modelo de programação que popularizou o processamento distribuído. Ele funciona em duas grandes etapas: a fase *Map*, na qual os dados são processados localmente em cada nó, e a fase *Reduce*, em que os resultados parciais são agregados. Embora extremamente robusto, o MapReduce apresenta limitações de performance, especialmente por depender fortemente de escrita e leitura em disco a cada etapa do processamento.

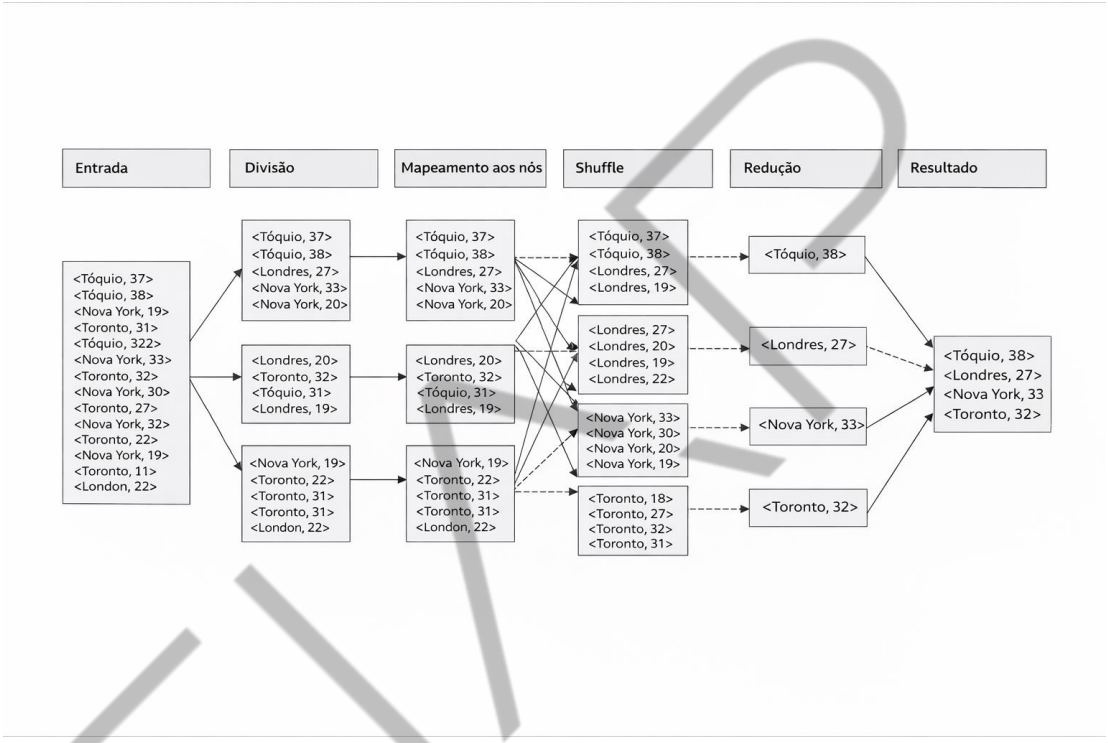


Figura 3 - MapReduce
Fonte: Elaborado pelo autor (2026)

O fluxo inicia-se na etapa de **Entrada (Input)**. Nessa fase, os dados são armazenados em um sistema de arquivos distribuído, como o HDFS, normalmente organizados em arquivos grandes. Esses arquivos contêm pares de dados que podem ser interpretados como **chave e valor**, por exemplo, registros de cidades associados a valores numéricos.

Em seguida, ocorre a **Divisão (Splitting)**. Os arquivos de entrada são automaticamente fragmentados em blocos menores chamados *splits*. Cada split representa uma parte do arquivo original e pode ser processado de forma independente. Essa divisão é fundamental para permitir o paralelismo, pois diferentes nós do cluster podem trabalhar simultaneamente sobre partes distintas dos dados.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Na etapa de **Mapeamento (Map)**, cada split é enviado para um nó de processamento, onde a função *Map* é executada. Essa função lê os dados de entrada e transforma cada registro em pares intermediários de chave e valor. O objetivo do Map não é agregar informações, mas sim preparar e organizar os dados para a próxima fase, mantendo a chave como elemento central do processamento.

Após o mapeamento, os dados passam pela fase de **Shuffle**, considerada uma das etapas mais críticas do MapReduce. Durante o shuffle, os pares intermediários gerados pelos diversos nós são redistribuídos pela rede. Todos os valores associados à mesma chave são agrupados e enviados para o mesmo nó responsável pela redução daquela chave específica. Esse processo envolve comunicação intensa entre os nós do cluster e é determinante para o desempenho da aplicação.

Na sequência, ocorre a etapa de **Redução (Reduce)**. Cada nó executa a função *Reduce* sobre os dados já agrupados pelo shuffle. Nessa fase, os valores associados a uma mesma chave são consolidados por meio de operações como soma, contagem, máximo ou mínimo, dependendo do problema a ser resolvido. O resultado do Reduce é um conjunto menor de dados, já agregado e pronto para consumo.

Por fim, chega-se à etapa de **Resultado (Result)**. Os dados processados e agregados são gravados novamente no sistema de arquivos distribuído. Esse resultado pode ser utilizado diretamente para análises, relatórios ou servir como entrada para novos processos de processamento distribuído.

Em conjunto, esse fluxo demonstra como o MapReduce divide o processamento em etapas bem definidas, explorando o paralelismo e a distribuição de dados para lidar com volumes massivos de informação de forma escalável e eficiente.

Já o **YARN (Yet Another Resource Negotiator)** atua como o gerenciador de recursos do cluster, controlando memória, CPU e a alocação de tarefas. Ele desacoplou o gerenciamento de recursos do modelo MapReduce, permitindo que outras engines de processamento pudessem coexistir no mesmo cluster Hadoop, abrindo caminho para a evolução do ecossistema.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Limitações do Hadoop tradicional

Apesar de sua importância histórica, o Hadoop baseado em MapReduce mostrou-se pouco flexível para cenários analíticos mais modernos. Processamentos iterativos, análises interativas e workloads de Machine Learning tornavam-se lentos devido à dependência excessiva de disco. Além disso, o desenvolvimento de aplicações MapReduce exigia maior complexidade de código, afastando usuários com perfil mais analítico.

Essas limitações não diminuem o valor do Hadoop, mas explicam por que novas ferramentas surgiram para complementar — e, em alguns casos, substituir — o modelo tradicional. É nesse contexto que o Apache Spark ganha protagonismo.

Spark: processamento distribuído em memória

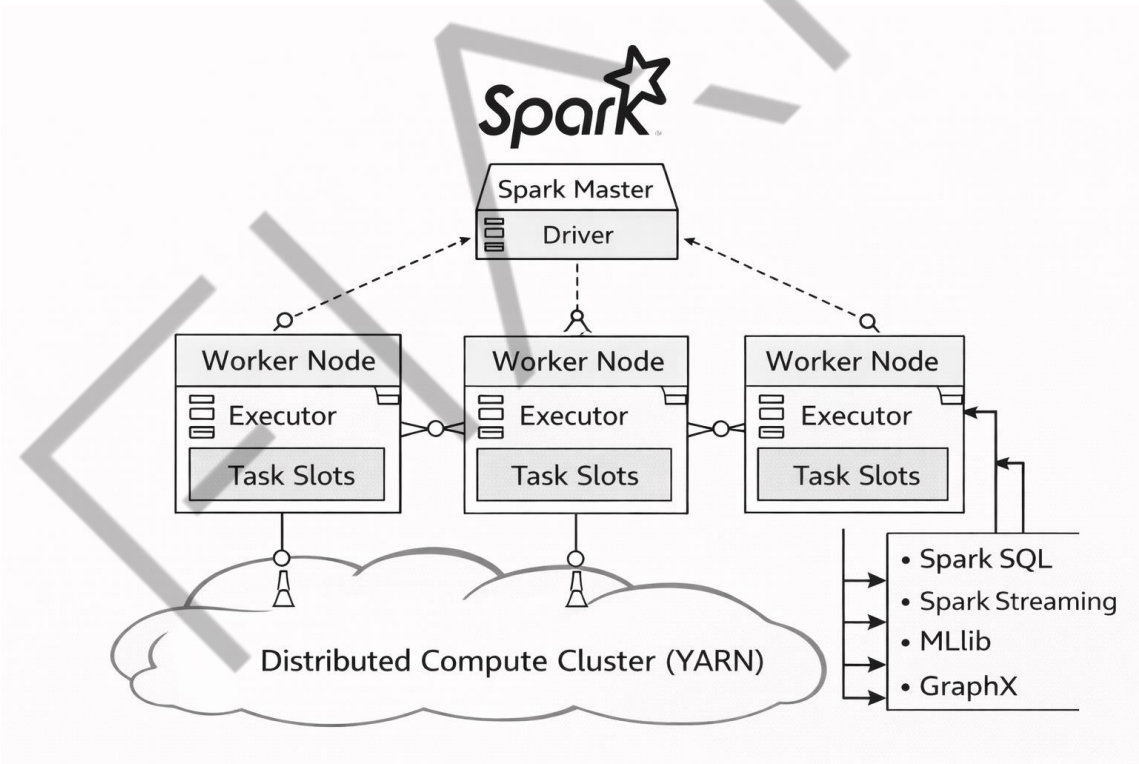


Figura 4 - Spark
Fonte: Elaborado pelo autor (2026)

O **Apache Spark** foi projetado para superar as principais limitações do MapReduce, trazendo uma abordagem mais moderna e eficiente para o processamento distribuído. Sua principal característica é o **processamento em**

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

memória, que reduz drasticamente o tempo de execução ao evitar acessos frequentes ao disco.

No Spark, os dados são representados por estruturas distribuídas chamadas RDDs e, mais recentemente, por **DataFrames e Datasets**, que oferecem uma API mais declarativa e otimizada. O motor do Spark constrói um **DAG (Directed Acyclic Graph)** de execução, analisando todo o fluxo de transformações antes de executar as tarefas. Isso permite otimizações automáticas, como reordenação de operações e redução de etapas desnecessárias.

Outro diferencial do Spark é sua versatilidade. Ele não se limita ao processamento batch, oferecendo módulos integrados para **Spark SQL**, **Structured Streaming**, **Machine Learning (MLlib)** e **processamento de grafos (GraphX)**. Essa unificação torna o Spark uma engine central para arquiteturas analíticas modernas.

Hadoop e Spark trabalhando juntos

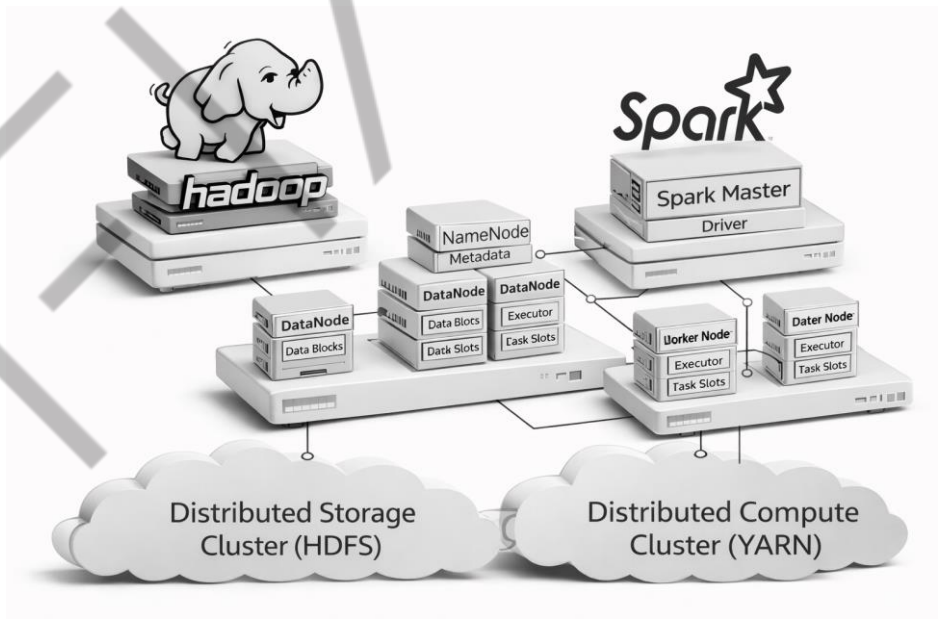


Figura 5 - Hadoop e Spark
Fonte: Elaborado pelo autor (2026)

É importante destacar que Hadoop e Spark não são tecnologias excludentes. Em muitos cenários, eles atuam de forma complementar. O HDFS continua sendo amplamente utilizado como camada de armazenamento distribuído, enquanto o Spark

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

assume o papel de motor de processamento. Nesse arranjo, o YARN pode gerenciar os recursos do cluster, permitindo que aplicações Spark sejam executadas lado a lado com outros workloads.

Essa integração é comum em ambientes corporativos, pois preserva investimentos existentes em Hadoop ao mesmo tempo em que incorpora ganhos significativos de performance e flexibilidade com Spark.

Do processamento distribuído ao Lakehouse

Com a evolução das arquiteturas de dados, o processamento distribuído passou a se integrar diretamente ao conceito de **Lakehouse**, no qual armazenamento e processamento coexistem de forma unificada. Plataformas modernas utilizam o Spark como engine principal para operar diretamente sobre dados armazenados em Data Lakes, eliminando a separação rígida entre Data Lake e Data Warehouse.

Nesse cenário, o processamento distribuído deixa de ser apenas um mecanismo técnico e passa a ser um **elemento estratégico da arquitetura**, viabilizando análises em larga escala, redução de custos e maior agilidade no ciclo de dados. Hadoop e Spark, juntos ou de forma independente, continuam sendo tecnologias fundamentais para sustentar esse modelo.

Por que dominar Hadoop e Spark é essencial em Big Data

Compreender Hadoop e Spark vai além de conhecer ferramentas específicas. Trata-se de entender **como sistemas distribuídos funcionam**, como dados são particionados, como falhas são tratadas e como o paralelismo é explorado de forma eficiente. Esses conceitos são reaproveitados em diversas outras tecnologias modernas de Big Data, computação em nuvem e engenharia de dados.

Ao longo desta disciplina, esses fundamentos servirão de base para a compreensão de pipelines escaláveis, arquiteturas analíticas modernas e soluções reais utilizadas pelo mercado. Revisite este conteúdo sempre que necessário e conecte-o com os exercícios práticos para consolidar o aprendizado.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

MERCADO, CASES E TENDÊNCIAS

Artigo: a **Netflix** utiliza o **Apache Spark** para análises em larga escala, experimentação A/B e sistemas de recomendação. A migração de MapReduce para Spark trouxe ganhos significativos de performance e produtividade.

Artigo**:** a **Uber** emprega Spark como engine central de processamento batch e streaming, lidando com trilhões de eventos e suportando modelos analíticos e operacionais em tempo quase real.

Case: a **Databricks**, criadora do conceito de Lakehouse, reúne diversos cases corporativos em que Hadoop e Spark são usados de forma integrada para substituir arquiteturas tradicionais fragmentadas.

Artigos técnicos de referência

Apache Hadoop vs Apache Spark — Databricks
Artigo comparativo que explica claramente as diferenças, complementaridades e a evolução do processamento distribuído. Link

Spark Architecture Overview — Databricks
Conteúdo técnico que detalha DAG, execução distribuída, processamento em memória e otimizações internas do Spark. Link

Livros recomendados (Inglês)

Learning Spark: Lightning-Fast Data Analytics
Matei Zaharia et al. O'Reilly Media, 2020.
Livro escrito pelos criadores do Spark, abordando fundamentos, arquitetura e aplicações práticas em ambientes produtivos.

Designing Data-Intensive Applications
Martin Kleppmann. O'Reilly Media, 2017.

Embora não seja exclusivo de Hadoop e Spark, é uma das obras mais importantes para compreender sistemas distribuídos, escalabilidade e processamento de dados em larga escala.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Livros recomendados (Português)

Big Data: Tecnologias e Aplicações

Eduardo Bezerra. Casa do Código, 2018. Apresenta conceitos fundamentais de Big Data, incluindo Hadoop, processamento distribuído e aplicações práticas no contexto corporativo.

Arquitetura de Big Data

Rafael Winterhalter. Novatec, 2019. Livro em português que contextualiza Hadoop, Spark e ecossistemas de dados distribuídos dentro de arquiteturas modernas.

Podcast especializado

Databricks Data Brew Podcast

Podcast oficial da Databricks com episódios focados em Spark, Lakehouse, Hadoop e casos reais de engenharia de dados.

Tendências do mercado

O mercado aponta para a consolidação do **Spark como engine padrão de processamento distribuído**, enquanto o Hadoop permanece relevante como ecossistema e camada de armazenamento. Arquiteturas Lakehouse, processamento em tempo real e integração com Machine Learning reforçam a importância do domínio desses fundamentos para profissionais de Big Data. Hadoop e Spark seguem não apenas como tecnologias, mas como **conceitos estruturais** que moldam as plataformas analíticas modernas.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você compreendeu por que o **processamento distribuído** é um elemento central nas arquiteturas de Big Data, tornando possível analisar grandes volumes de dados de forma escalável, resiliente e eficiente. Foi discutido como a divisão de dados e tarefas entre múltiplos nós de um cluster permite explorar paralelismo e tolerância a falhas, superando as limitações do processamento centralizado.

Você também conheceu os fundamentos do **Apache Hadoop**, entendendo o papel do HDFS no armazenamento distribuído, do YARN no gerenciamento de recursos e do modelo MapReduce como marco histórico do processamento em larga escala. Em seguida, a aula avançou para o **Apache Spark**, destacando seu processamento em memória, o uso de DAGs, DataFrames e sua versatilidade para workloads analíticos, streaming e Machine Learning.

Por fim, a integração entre Hadoop e Spark foi contextualizada dentro de arquiteturas modernas, especialmente no modelo Lakehouse, conectando armazenamento em Data Lakes, processamento distribuído e camadas analíticas. O hands on com Databricks consolidou esses conceitos ao mostrar, na prática, como dados são lidos, transformados e consumidos em escala, reforçando a visão de ponta a ponta da Arquitetura de Big Data.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

REFERÊNCIAS

APACHE SOFTWARE FOUNDATION. **Apache Hadoop.** Disponível em: https://hadoop.apache.org/. Acesso em: 26 fev. 2026.

APACHE SOFTWARE FOUNDATION. **Apache Spark Documentation.** Disponível em: https://spark.apache.org/docs/latest/. Acesso em: 26 fev. 2026.

BREWER, Eric A. **Towards Robust Distributed Systems.** Proceedings of the Annual ACM Symposium on Principles of Distributed Computing, 2000.

DATABRICKS. **Apache Hadoop vs Apache Spark**. Disponível em: https://www.databricks.com/glossary/hadoop-vs-spark. Acesso em: 26 fev. 2026.

DATABRICKS. **Spark Architecture Overview**. Disponível em: https://www.databricks.com/spark/architecture. Acesso em: 26 fev. 2026.

KLEPPMANN, Martin. **Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems**. Sebastopol: O'Reilly Media, 2017.

NETFLIX TECHNOLOGY BLOG. **Apache Spark at Netflix**. Disponível em: https://netflixtechblog.com/apache-spark-at-netflix-2b3c4b8fdb77. Acesso em: 26 fev. 2026.

UBER ENGINEERING. **Spark at Uber**. Disponível em: https://www.uber.com/blog/spark-at-uber/. Acesso em: 26 fev. 2026.

WHITE, Tom. **Hadoop: The Definitive Guide**. 4. ed. Sebastopol: O'Reilly Media, 2015.

ZAHARIA, Matei; XIN, Reynold S.; WENDELL, Patrick; DAS, Tathagata et al. Learning **Spark: Lightning-Fast Data Analytics**. 2. ed. Sebastopol: O'Reilly Media, 2020.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

PALAVRAS-CHAVE

Processamento distribuído. Apache Spark. Hadoop.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com



PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com